

SDP/PDS OS Internals

Date: 1/9/2025

Questions

Theoretical Part

Question 1

A) Consider the following statements regarding the possible advantages and disadvantages of an inverted page table (IPT) compared to a standard page table (PT). Tell each one whether it is true or false (YES/NO):

1. The IPT reduces the address space of processes.
2. If the size of a process' address space is equal to half the RAM size, the process' PT is generally smaller than the IPT.
3. If the size of a process' address space is equal to the RAM size (excluding the RAM allocated to the kernel), the process' PT and the IPT can have equal number of entries.
4. In the presence of multiple processes, each one with its own PT, an IPT would be a better alternative in terms of memory usage, because each process' address spaces is at least large as the RAM.
5. When the IPT is combined with a hash table, the size is doubled, because IPT entries are replicated in the hash table.
6. The hash table improves search time because the page number is the index in the hash array.

B) Consider a process P1, having a virtual addressing space of 72 GB, in a system equipped with 32GB of RAM, with 64-bit architecture (in which the Byte is addressed) and paged memory management (4KB pages/frames) for user processes only, so not for the kernel. Assume 8GB of RAM are statically allocated to the kernel. You want to compare a standard page table-based solution (one table for each process) and an IPT-based solution. Calculate:

- The number of entries in the Page Table.
- The number of entries in the IPT, assuming that the RAM allocated to the kernel is not mapped by the IPT.
- The minimum number of bits required for a frame number, assuming that the system could support RAM sizes up to 256GB.

- The minimum number of bits required for a page number, assuming that the system could support address spaces up to 4TB.
- The size of an IPT entry, assuming that a PID has to be represented on 12 bits, 6 bits are needed for flags, and the size is a power of 2.
- The ratio between the PT size and the IPT size, assuming that a PT entry needs to include 6 flag bits, and the size of the entry is a power of 2.

Question 2

A) A disk organized with RAID structure is given. Consider the mean time between failures (MTTF), the mean time to repair (MTTR), and the mean time between data loss (MTTDL).

- Can mirroring produce a MTTDL less than the MTTF (assume $MTTR \ll MTTF$)?
- Can striping reduce the MTTF of a given disk?

B) Consider a RAID structure with 4 disks in a "striped" configuration (RAID 0: non-redundant striping). Every disk has $MTTF=10000$ hours. Compute the MTTDL (considering a partial data loss, on one of the 4 disks).

C) Consider a RAID structure with 4 disks in a "striped+mirrored" configuration (striping on 2 disks, then mirroring with another 2 disks). If each of the disks can fail independently of the other, with $MTTF = 10000$ hours, compute the MTTR to ensure $MTTDL \geq 100 \times MTTF$?

Question 3

A) Consider disk scheduling algorithms. Answer the following questions (YES/NO):

- The C-SCAN algorithm optimizes positioning time for HDDs, by minimizing rotational latency.
- Given the same set of disk service requests, the total travelling distance of the disk head in C-SCAN is generally higher than in SCAN.
- NVM (Non-Volatile Memory) disks do not require the same scheduling algorithms used for HDDs.
- Given a disk request queue with N requests, no additional request can be accepted (and enqueued) while an STTF (Shortest Seek Time First) algorithm is servicing the N requests.
- Seek time minimization is the target of scheduling algorithm for HDDs.

B) A large text file contains N lines of fixed length (40 chars for each line), containing a set of sorted names. A binary (dichotomic) search algorithm is implemented directly on the file (either exploiting memory mapping of files or by properly using seek, read and write primitives). Depending on the underlying file system (file allocation strategy: contiguous, linked list, FAT or based on Inodes), we expect that (YES/NO):

- All allocation strategies will provide a logarithmic cost, as binary search is logarithmic and the algorithm and the allocation are independent each other, and operating at different SW layers.
- Contiguous allocation guarantees a logarithmic cost, as random/direct access is supported with constant time ($O(1)$).
- FAT and linked list do not support $O(1)$ random/direct access, so binary search becomes $O(N \log N)$: as a consequence linear search is better in term of complexity.
- Inodes do not support random/direct access, so they do not guarantee logarithmic binary search.
- Though FAT only support sequential access, it can be much more efficient than linked lists with binary search.

OS161

Question 4

Consider, in OS161, the implementation of virtual memory management. The `as_copy` function aims to "duplicate" an address space, i.e. to create a new address space, starting from an existing one, of which a copy is made. The function could be used, for example, to realize the system call `fork`, which generates a child process starting from a copy of an existing process (parent).

A possible implementation of `as_copy` is proposed:

```
1 int as_copy(struct addrspace *old, struct addrspace **ret) {
2     struct addrspace *new;
3
4     dumbvm_can_sleep();
5     new = as_create();
6
7     new->as_vbase1 = old->as_vbase1;
8     new->as_npages1 = old->as_npages1;
9     new->as_vbase2 = old->as_vbase2;
10    new->as_npages2 = old->as_npages2;
11
12    if (as_prepare_load(new)) {
13        as_destroy(new);
14        return ENOMEM;
15    }
16
17    memmove((void *) PADDR_TO_KVADDR(new->as_pbase1),
18            (const void *) PADDR_TO_KVADDR(old->as_pbase1),
19            old->as_npages1*PAGE_SIZE);
20
21    memmove((void *) PADDR_TO_KVADDR(new->as_pbase2),
22            (const void *) PADDR_TO_KVADDR(old->as_pbase2),
23            old->as_npages2*PAGE_SIZE);
24
25    memmove((void *) PADDR_TO_KVADDR(new->as_stackpbase),
26            (const void *) PADDR_TO_KVADDR(old->as_stackpbase),
27            DUMBVM_STACKPAGES*PAGE_SIZE);
28
29    *ret = new;
30    return 0;
31 }
```

Answer the following questions (or say YES/NO for true/false sentences):

- Is the function allocating physical RAM memory, when calling `as_prepare_load`?
- The explicit transformations from physical to logical address (e.g. `PADDR_TO_KVADDR(new->as_pbase1)`) could be avoided by simply using logical addresses (e.g. `new->as_vbase1`).
- If the `fork` system call is implemented using `as_copy`, the copy-on-write technique cannot be supported.

- The declaration of the parameter `ret`, using a double star, is an error: there should be a single star, and the last statement should be `ret = new`;
- The generated address space (pointed by `new`) is a user data structure, seen by a process when running in user mode.

Given the following values for `new->as_vbase1`, `new->as_pbase1`, `new->as_npages1`, `new->as_vbase2`, `new->as_pbase2`, `new->as_npages2`: 0x400000, 0x200000, 2, 0x480000, 0x300000, 4

- Which is the result of `PADDR_TO_KVADDR(new->as_pbase1)`?
- Which is the physical address for logical address 0x4000AB?
- Which is the kernel logical address for physical address 0x303000?

Question 5

Consider a user process on OS161. For each of the following sentences, say YES/NO (if they are true/false):

- The switchframe of a process is in its kernel stack and it is dynamically allocated (with `kmalloc`).
- The `switchframe_switch` function is implemented in assembly code: it switches context by saving the context of a thread and restoring the context of another thread.
- The user stack of a process is not part of its address space, which is limited to the code and data segments.
- In Dumbvm the user stack of a process has a fixed size of 18 pages.
- The `enter_new_process` function is used to start a user process by creating a trapframe and simulating a return from exception.
- The `enter_new_process` function waits for the process termination and returns the exit status of the user process.
- When a user process performs a `read(fd, buf, nb)`, `buf` is the source and `fd` the destination.
- When a user process performs a `read(fd, buf, nb)`, `buf` can be a pointer to a kernel buffer (if kernel buffering is enabled).
- The MIPS system call handler in OS161 (function `syscall`) receives all needed data in the trapframe: so the function has only one parameter, the pointer to a trapframe.
- When the `syscall` function terminates, it signals (function `V`) a semaphore on which the user process is waiting for the system call to terminate.